MINI PROJECT REPORT

ON

"Mochi: A Persistent Electron-Based Desktop Virtual Pet System with Real-Time State Management"

Under the Course
Web Technology

SUBMITTED BY

PRN	NAME OF STUDENT
1012411001	Riddhi Narendra Chatorikar
1012411009	Mrudula Vijay Pimparwar
1012411012	Lobhas Mahesh Kulkarni

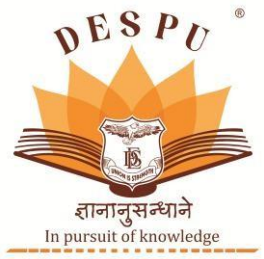
2026-27

Third Year Computer Science & Engineering (TY CSE)

Division: A, Semester I

Under the Guidance Of
DR. NEERAJ KALE

Department of Computer Engineering and Technology
Program: B.Tech Computer Science & Engineering
School of Engineering and Technology
DES Pune University, Pune.



CERTIFICATE

This is to certify that students Riddhi Narendra Chatorikar, Mrudula Vijay Pimparwar, and Lobhas Mahesh Kulkarni, PRN. 1012411001, 1012411009, 1012411012 are studying in the **TY CSE (Division: A)** course in SEM I, and they have successfully completed and submitted the **Mini Project Report under the Course Web Technology**, entitled "Mochi: A Persistent Electron-Based Desktop Virtual Pet System with Real-Time State Management".This study is a partial fulfilment of the degree of Bachelor of Technology in Computer Science and Engineering of **DES Pune University, Pune** during the academic year **2026-2027.**

DR. NEERAJ KALE — Guide

Dr. PRACHI JOSHI — HoS

ABSTRACT

Mochi: A Persistent Electron-Based Desktop Virtual Pet System with Real-Time State Management is an interactive, cross-platform desktop application that allows users to adopt, name, and care for a digital pet that lives directly on their desktop. Users choose from four pet types (cat, dog, rabbit, and elephant) and perform pet-care activities such as feeding, playing, sleeping, and bathing. Each activity affects the pet's condition, represented through attributes including hunger, energy, happiness, cleanliness, and health. The project focuses on continuity: the pet retains its state when the application is closed and relaunched.

The system is built with Electron, which packages standard web technologies as a native desktop application. HTML defines the structure of the interface, CSS styles the screens, and JavaScript implements the game logic. The Electron main process (main.js) manages the native window, desktop-pet (mini) mode, the right-click context menu, the always-on-top setting, and a single-instance lock, while the renderer process runs the user interface. The two communicate through Inter-Process Communication (IPC) using ipcMain, ipcRenderer, and a secure contextBridge exposed by a preload script. Pet information is saved with LocalStorage inside Electron's embedded Chromium and restored on the next launch. Pet statistics change in real time while the application runs, with each pet type having its own rates and traits. All pet artwork is generated programmatically as inline SVG, so the application needs no external image assets. The design also includes a food shop and pantry, a "Catch the Ball" mini-game, and progression through experience points (XP), coins, and levels.

The project demonstrates how web technologies can be combined with a desktop runtime to create a responsive, interactive application with persistent state and native-feeling behaviour such as a draggable, always-on-top floating pet. It provides a practical learning example of event handling, IPC between Electron processes, local data persistence, programmatic SVG graphics, and timer-driven application logic. The report describes the project motivation, objectives, system scope, architecture, working, implementation approach, results, limitations, and possible future enhancements.

Keywords: contextBridge, Desktop application, Electron, HTML, IPC, JavaScript, LocalStorage, persistent state, real-time state management, SVG, virtual pet

INDEX

Sr.No	Chapters	Page Numbers
1	Introduction	5 – 6
2	Problem Statement & Objectives	7 - 8
3	Existing System & Proposed Solution	9 - 10
4	Project Scope & Requirements	11 - 12
5	System Design & Architecture	13 - 15
6	Implementation & Working	16 - 19
7	Results & Testing	20 - 21
8	Conclusion & Future Scope	22 - 23
9	References	24

Chapter 01: Introduction

1.1 Background

Interactive applications are commonly used for entertainment, learning, and engagement. A virtual pet is a digital companion whose condition changes according to user actions and the passage of time. Users can perform simple care activities and observe how those activities influence the pet's needs and mood. Unlike a static interface, a virtual pet application maintains a changing state and provides feedback after each interaction.

Electron allows an application to be written with HTML, CSS, and JavaScript while running as a native desktop program on Windows, macOS, and Linux, by embedding the Chromium rendering engine and the Node.js runtime. A desktop implementation lets the pet live outside a browser tab: it can shrink into a small, frameless, transparent window that floats above other applications. However, an application that stores its state only in temporary variables loses its progress when it is closed. This project addresses that limitation by saving pet data with LocalStorage inside Electron and by updating the pet's condition in real time through a periodic timer.

1.2 Project Overview

Mochi is a desktop virtual pet. The user adopts a pet (cat, dog, rabbit, or elephant), names it, views its status, and performs activities such as feeding, playing, sleeping, and bathing. The system tracks hunger, energy, happiness, cleanliness, and health, which change over time at rates that depend on the pet type. When an activity is performed, the relevant attributes are updated and the interface reflects the new condition. Food can be bought with coins in a shop and stored in a pantry, and a "Catch the Ball" mini-game lets the user earn coins. The application supports progression through XP, coins, and levels. Persistence is central to the project: the pet's state is saved to LocalStorage and restored when the application is relaunched. The application can also be minimized into a tiny floating desktop pet that is draggable, stays on top of other windows, and offers a right-click menu. The complete source code is available at https://github.com/RiddhiInLoop/Mochi_VP.

1.3 Need for the Project

The project is useful as a compact demonstration of modern desktop application development using web technologies. It brings together interface design, user-triggered events, state updates, local persistence, timer-driven behaviour, programmatic SVG graphics, and communication between Electron's main and renderer processes in one application. It also illustrates how a small game-like experience can encourage repeated engagement by giving users a pet to care for and progress to maintain.

Chapter 02: Problem Statement & Objectives

2.1 Problem Statement

Traditional interactive applications and browser games often lose progress when the page is refreshed or the browser is closed, and a pet confined to a browser tab cannot stay visible while the user works in other applications. In a virtual pet application, losing the pet's previous condition breaks continuity and weakens the feeling that the pet exists beyond a single session. A static status display also cannot represent how a pet's needs change as time passes. The problem is to develop a desktop virtual pet system that preserves pet information across launches, responds to care activities, updates the pet's condition in real time, and can live on the desktop as a small floating companion. The interface should be clear and easy to use, while the application should provide a consistent way to save, restore, and update the pet's state.

2.2 Objectives

1. To develop an interactive virtual pet application that runs as a cross-platform Electron desktop application.
2. To support virtual pet adoption and naming, with four pet types (cat, dog, rabbit, and elephant) that have different stat rates and traits.
3. To implement feeding, playing, sleeping, and bathing as pet-care activities.
4. To track hunger, energy, happiness, cleanliness, and health.
5. To update pet attributes in response to user actions.
6. To save and restore pet information using LocalStorage within Electron.
7. To update pet attributes continuously in real time using a periodic tick, with pet-specific rates and modifiers.
8. To provide a clear, user-friendly interface with visual feedback for every action.
9. To introduce progression through XP, coins, and levels.
10. To provide a food shop and pantry inventory, and a "Catch the Ball" mini-game that rewards coins.
11. To implement a desktop-pet (mini) mode with a draggable, always-on-top floating pet and a right-click context menu, using Electron's main process and IPC (ipcMain, ipcRenderer, contextBridge).

12. To generate all pet artwork programmatically as inline SVG, without external image assets.
13. To package the application for Windows, macOS, and Linux using electron-builder.

2.3 Expected Outcome

The intended outcome is a working Electron desktop application in which a user can interact with a pet, see the results of care actions, close and relaunch the application, and find the pet restored. The pet's condition changes in real time according to the implemented tick rules, and the pet can be minimized into a floating desktop companion.

Chapter 03: Existing System & Proposed System

3.1 Existing System

A simple browser game may keep its data only in JavaScript variables while the page is open. This approach is easy to implement, but a page refresh or a new browser session can reset the application. Such a pet also lives only inside a browser tab, so it cannot remain visible on the desktop while the user works in other applications. Games that depend on image files additionally require external assets to be loaded and packaged. These limitations can make a virtual pet feel like a temporary interface rather than a continuing digital companion.

3.2 Limitations of the Existing Approach

1. Pet status or progress may be lost between sessions.
2. The pet lives inside a browser tab and cannot stay on the desktop alongside other work.
3. User actions may have limited or inconsistent effects on pet attributes, and all pets may behave identically.
4. Image-based artwork requires external assets, which increases dependencies and package size.
5. Without progression and reward elements (XP, coins, mini-games), repeated interaction may offer limited motivation.

3.3 Proposed System

The proposed system, Mochi, combines interactive pet care with persistent local storage and real-time state management inside an Electron desktop application. The user can adopt and name a pet, view its attributes, buy food, and perform care activities. JavaScript in the renderer process applies the relevant state changes and updates the interface, while the Electron main process handles native window behaviour such as mini mode, the context menu, and always-on-top. LocalStorage stores the pet's data so that it can be restored at the next launch, and a periodic tick keeps the pet's condition changing while the application is running.

3.4 Advantages of the Proposed System

1. Continuity of pet data across application restarts through LocalStorage.
2. Interactive care actions with visible changes in pet attributes.
3. Real-time stat changes with pet-specific rates, giving each of the four pet types its own character.
4. A dashboard-style interface for monitoring the pet's condition.
5. XP, coins, and levels to support progression, with a food shop, pantry, and mini-game to earn and spend coins.
6. A desktop-pet mode with a draggable, always-on-top floating pet and a right-click menu, made possible by Electron's main process and IPC.
7. All pet artwork is generated as pure SVG, so no external image assets are needed, and the application can be packaged for Windows, macOS, and Linux with electron-builder.

Chapter 04: Project Scope & Requirements

4.1 Functional Scope

1. Pet adoption and naming: allow the user to begin by choosing one of four pet types (cat, dog, rabbit, or elephant) and naming it.
2. Status monitoring: show hunger, energy, happiness, cleanliness, and health.
3. Care interactions: provide feeding, playing, sleeping, and bathing actions.
4. State updates: modify pet attributes in response to actions.
5. Persistence: save the current pet state and restore it when the application is relaunched.
6. Real-time behaviour: a periodic tick changes the pet's stats, with pet-specific modifiers (for example, the dog and elephant get hungry faster and the rabbit loses energy faster).
7. Progression: provide XP, coins, and levels.
8. Food shop and pantry: buy food with coins, store it in a pantry inventory, and feed the pet.
9. Mini-game: play "Catch the Ball" to earn coins and increase the pet's happiness.
10. Desktop-pet mode: minimise to a small, draggable, always-on-top floating pet with a wander animation and a right-click context menu.

4.2 Technical Scope

The project is an Electron desktop application. Electron combines the Chromium rendering engine and the Node.js runtime. The main process (main.js) creates and manages the native window and handles IPC messages; a preload script (preload.js) exposes a limited set of functions to the renderer through contextBridge; and the renderer process (index.html, style.css, app.js, pets.js) provides the interface and game logic. HTML is used for the interface structure, CSS for layout and styling, and JavaScript for event handling, game rules, attribute updates, drag behavior, and data persistence. All pet art is generated as inline SVG by pets.js. LocalStorage, provided by the embedded Chromium, is used as the storage mechanism, and electron-builder is used to package the application.

4.3 Software Requirements

1. Node.js (version 18 or newer recommended) and npm.
2. Text editor or code editor for development.
3. Electron framework (main process, preload script, and renderer process).
4. HTML and CSS for the interface structure and presentation.
5. JavaScript for application logic, SVG generation, and state management.
6. electron-builder for packaging the application (portable .exe on Windows, .dmg on macOS, .AppImage on Linux).

4.4 Hardware Requirements

The application is designed to run on a general-purpose computer running Windows, macOS, or Linux that is capable of running an Electron application. A standard display, keyboard, and mouse are adequate. No specific minimum processor, RAM, or storage values have been defined or measured for this project.

4.5 Project Boundaries

The current scope focuses on a local, single-user desktop application. In Electron, LocalStorage data is tied to the application's own data directory rather than to a browser profile, so clearing the data of a web browser such as Chrome does not affect the saved pet, whereas deleting the application's data or using its reset option does. The data does not automatically provide cloud synchronization or shared accounts. Cloud-based accounts, native notifications, achievements, offline time simulation, and smarter pet behaviour are considered future enhancements rather than current features.

Chapter 05: System Design & Architecture

5.1 Architecture Overview

The system is built as an Electron application with a clear split between two processes. The user interacts with the renderer process, which runs the HTML, CSS, and JavaScript interface, the game logic, and the SVG pet art. When a native action is needed, such as minimizing to desktop-pet mode, showing the right-click menu, or toggling always-on-top, the renderer sends a message through the contextBridge exposed by the preload script, which uses IPC to reach the main process. The main process (Node.js) controls the native window on the operating system, including the frameless, transparent, always-on-top window used in mini mode. Pet data is saved to and loaded from LocalStorage provided by the Chromium engine embedded in Electron, which is stored in the application's data directory.

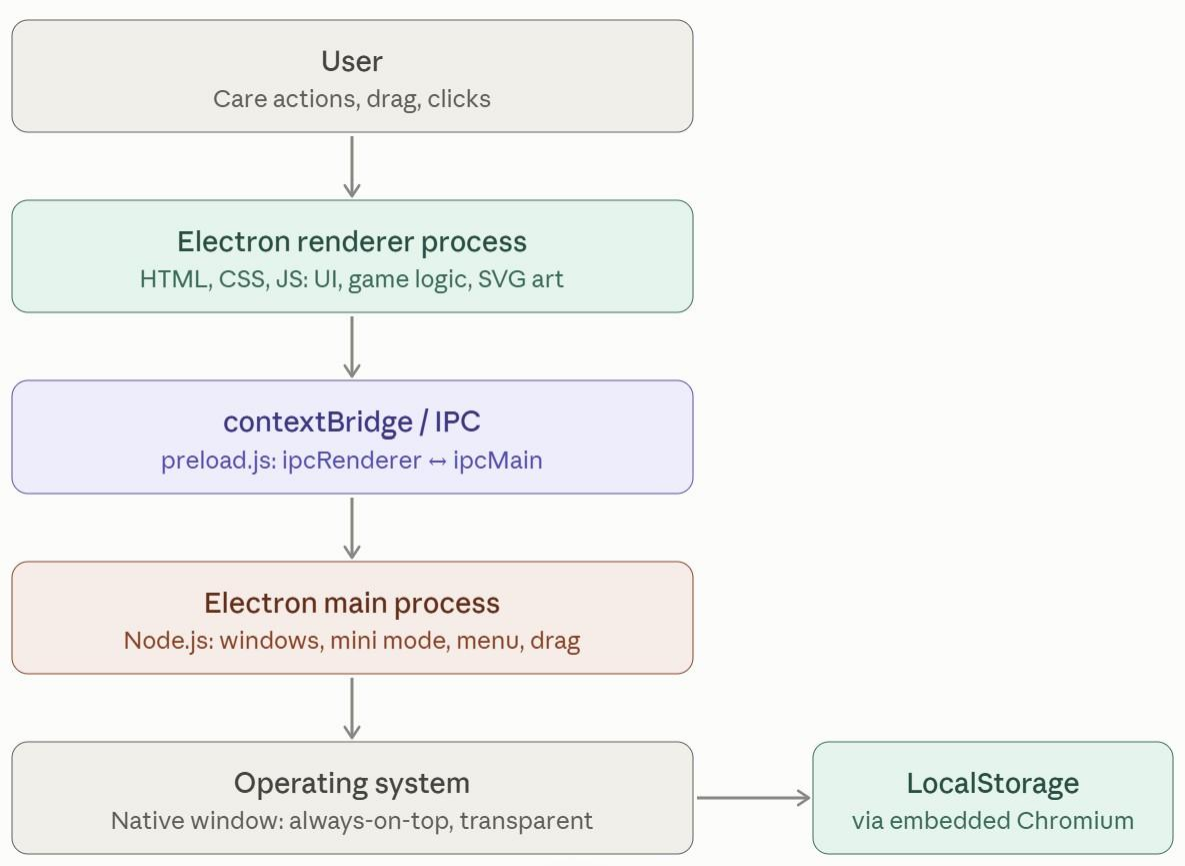


Figure 1. Architecture of the Mochi desktop pet system, showing the Electron renderer/main process split.

5.2 Main Modules

a) User Interface Module: This module, running in the renderer process, presents the pet and its current status through the application's screens. It provides controls for care activities and displays changes in the pet's attributes, XP, coins, and level.

b) Pet-State Management Module: This module maintains the active pet data. Attributes such as hunger, energy, happiness, cleanliness, and health represent the current condition. User actions trigger updates according to the game rules, and each pet type (cat, dog, rabbit, elephant) applies its own modifiers.

c) Real-Time Update Module: This module runs a periodic tick() function that changes the pet's attributes while the application is running. Pet-specific rates apply: the dog gets hungry 30% faster, the elephant's hunger drains 50% faster, the rabbit's energy drains 60% faster, and the cat is balanced. A wander timer (setInterval every 4 seconds) also moves the pet around, and a sleeping pet recovers energy at regular intervals.

d) Persistence Module: The persistence module uses LocalStorage, provided by Electron's embedded Chromium, to save the pet state in the application's data directory. On the next launch, the application reads the stored information and restores the previous state.

e) Progression Module: Progression features include experience points (XP), coins, and levels. Every interaction earns XP, and coins are earned in the mini-game and spent in the food shop.

f) Electron Main Process and IPC Module: main.js creates the window, enforces a single-instance lock (app.requestSingleInstanceLock()), and handles IPC messages through ipcMain, for example 'mini' (switch to desktop-pet mode) and 'menu' (show the right-click context menu). It toggles always-on-top with win.setAlwaysOnTop(). preload.js uses contextBridge to expose only the required functions to the renderer through ipcRenderer.

g) Shop and Mini-Game Module: This module implements the food shop and pantry inventory, in which food is bought with coins and then fed to the pet, and the "Catch the Ball" mini-game, which has a full game loop (startGame()) and rewards coins and happiness.

h) SVG Art Module: pets.js defines the pet types and generates all pet artwork programmatically as inline SVG, so no image files are used.

5.3 Data Flow

1. The user launches the Mochi desktop application; Electron creates the window and a single-instance lock prevents duplicate copies.
2. The renderer process checks LocalStorage for previously saved pet data.
3. If data exists, the application restores it; otherwise, the user begins the pet adoption flow by choosing a pet type and naming the pet.
4. The periodic tick starts and updates hunger, energy, happiness, cleanliness, and health using the pet-specific rates.
5. The dashboard displays the pet's current attributes and available actions.
6. The user performs a care activity (feeding from the pantry, playing the mini-game, sleeping, or bathing), and JavaScript updates the appropriate attributes, XP, and coins.

The updated state is saved to LocalStorage. Native actions such as mini mode, the context menu, and always-on-top are sent to the main process through contextBridge and IPC.

Chapter 06: Implementation & Working

6.1 Technologies Used

HTML: HTML (src/index.html) defines the structure of the application shell, which holds all of the application's screens, including the pet display, status indicators, action controls, shop, and mini-game areas.

CSS: CSS (src/css/style.css) controls the visual appearance of the application, including spacing, colours, typography, and layout. All styling is hand-written, without UI libraries.

JavaScript: JavaScript (src/js/app.js) provides the interactive behaviour. It handles user events, applies care-action rules, updates pet attributes through the tick, manages progression, runs the food shop and mini-game, implements Pointer Events drag in desktop-pet mode, and coordinates saving and restoring data.

LocalStorage: LocalStorage is a Web Storage API that stores key-value data persistently. In Electron, it is provided by the embedded Chromium and its data is kept in the application's own data directory, not in a browser profile. In this project it retains pet information between launches. Since LocalStorage stores string values, structured data is serialized before saving and parsed when loading.

Electron: Electron is a framework that combines Chromium and Node.js to build cross-platform desktop applications using web technologies. It provides the main process, which controls native windows and the operating system, and renderer processes, which display the user interface.

IPC and contextBridge: Inter-Process Communication (IPC) allows the renderer and main processes to exchange messages. The renderer uses ipcRenderer, the main process listens with ipcMain, and preload.js uses contextBridge to expose a safe, limited API to the renderer instead of giving it direct access to Node.js.

SVG: Scalable Vector Graphics (SVG) is an XML-based vector image format. In this project, pets.js generates all pet artwork programmatically as inline SVG, so the application needs no image files and the art scales cleanly.

electron-builder: electron-builder packages the application for distribution: a portable .exe for Windows, a .dmg for macOS, and an. AppImage for Linux.

6.2 Working of Care Activities

The application includes four principal care actions, together with a desktop-pet mode. Each action is intended to influence one or more pet attributes.

1. **Feeding:** opens the food shop and pantry. Food is bought with coins and then fed to the pet to reduce hunger (the rabbit gets 1.5× benefit from carrots).
2. **Playing:** launches the "Catch the Ball" mini-game, which earns coins and makes the pet happier.
3. **Sleeping:** puts the pet to sleep so that energy recovers (+5 every few seconds) until the user wakes it.
4. **Bathing:** instantly increases cleanliness by 40.
5. **Desktop-pet mode:** the minimize button shrinks the application into a tiny floating pet that is draggable, stays on top of other windows, and wanders about; a right-click menu opens the main window, toggles always-on-top, or quits.

6.3 State Persistence

When pet data changes, the current state is serialized and written to LocalStorage. The data that is retained includes the pet's type and name, attribute values, and progression information such as XP, coins, and level. On application startup, the saved record is retrieved and parsed. If saved data is unavailable, the application starts the new pet adoption flow. Because Electron stores LocalStorage in the application's data directory, the saved pet survives application restarts and is not affected by clearing a web browser's data. The reset (✕) option releases the pet and clears the save data.

6.4 Real-Time State Management

A living pet should respond not only to direct user actions but also to the passage of time. To support this, the application runs a periodic tick () function that adjusts hunger, energy,

happiness, cleanliness, and health while the application is running. Each pet type has its own modifiers: the dog gets hungry 30% faster, the elephant's hunger drains 50% faster, the rabbit's energy drains 60% faster (but it gets 1.5× benefit from carrots), and the cat has balanced stats. A separate timer (setInterval every 4 seconds) makes the pet wander. The stats are updated in real time only while the application is open; simulating the time elapsed while the application is closed is not implemented and is listed under future scope.

6.5 Electron Architecture and IPC

The application is divided into the main process and the renderer process. The main process (main.js) runs in Node.js, creates the native window, enforces a single-instance lock with app.requestSingleInstanceLock(), and registers IPC handlers with ipcMain (for example 'mini' for desktop-pet mode and 'menu' for the right-click context menu). It also toggles the always-on-top state with win.setAlwaysOnTop(). The renderer process runs the interface and game logic. For security, preload.js uses contextBridge to expose only the required functions, which call ipcRenderer to send messages to the main process.

6.6 Programmatic SVG Art

All pet artwork is generated in code. The file pets.js defines the pet types and contains the functions that build each pet's inline SVG, so the application uses no external images. This keeps the package small, lets the art scale to any size (including the tiny mini-mode pet), and allows expressions and states to change dynamically.

6.7 Desktop-Pet (Mini) Mode

When the user clicks the minimize button, the renderer sends an IPC message and the main process shrinks the window into a small, frameless, transparent, always-on-top window. The floating pet can be dragged anywhere on the screen using Pointer Events implemented in app.js, and it wanders around using a timer. Right-clicking the pet sends a 'menu' message to the main process, which shows a context menu to open the main window, toggle always-on-top, or quit.

6.8 User Experience and Progression

A clear dashboard allows users to understand the pet's condition and choose a suitable care action. Visual feedback after an action helps communicate cause and effect. XP, coins, and levels

add a progression layer, and the food shop and “Catch the Ball” mini-game give users goals and a way to earn and spend coins beyond maintaining the pet’s basic needs.

6.9 Visualization of the Project

Figure 2 gives a visual overview of the Mochi application. It shows the main screens in the order a user meets them: adoption and naming, the main pet room with the status bars, the food menu, the “Catch the Ball” mini-game, and the sleep screen. It also shows the minimized, draggable desktop-pet mode and the different pet expressions (idle, walking, sleeping, eating, and playing).

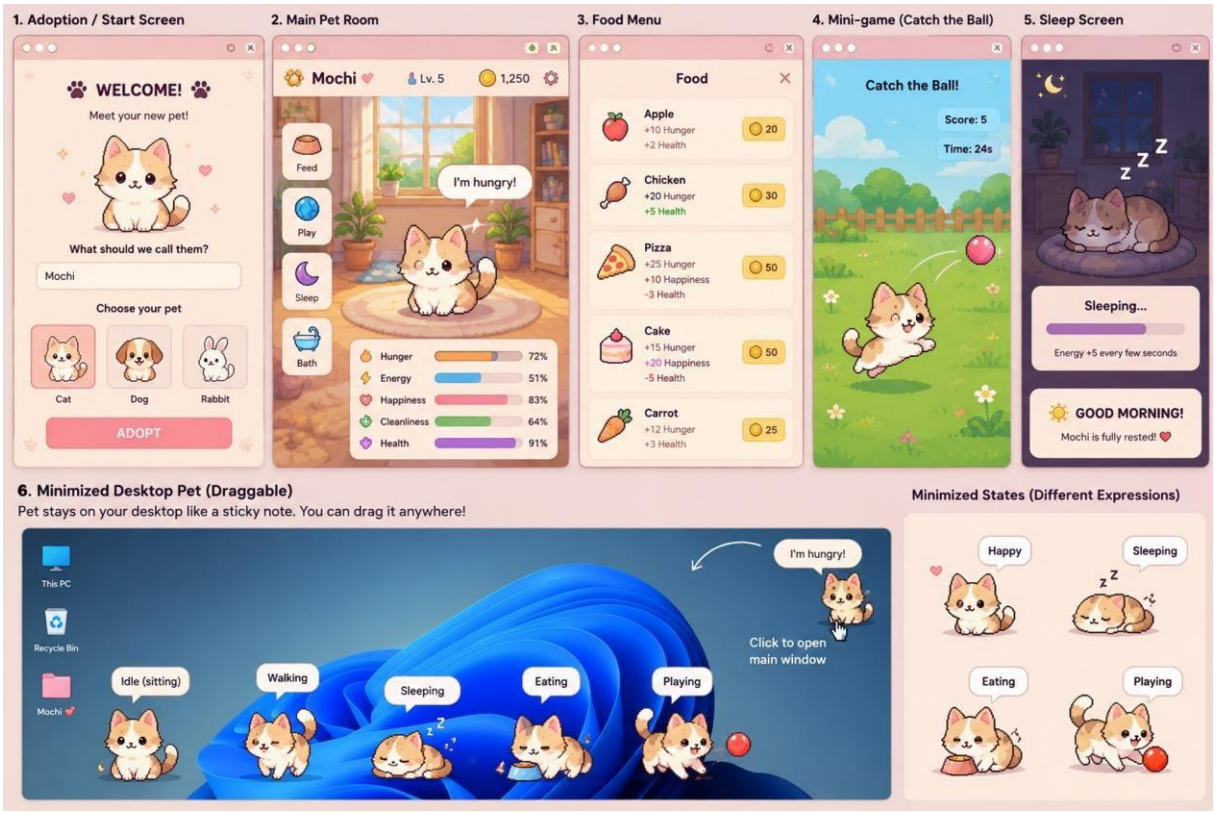


Figure 2. Visual overview of the Mochi application: main screens and minimized desktop-pet mode.

Chapter 07: Results & Testing

7.1 Intended Results

Based on the implemented application (source code: https://github.com/RiddhiInLoop/Mochi_VP), the system provides an interactive virtual pet dashboard, care actions that modify attributes, a dynamic display of pet mood and condition, persistent storage through LocalStorage, restoration of saved state between launches, real-time stat updates with pet-specific rates, a food shop and mini-game, progression through XP, coins, and levels, and a desktop-pet mode.

1. An interactive dashboard for viewing pet information.
2. Pet attributes that respond to care actions.
3. Pet stats that change in real time at pet-specific rates.
4. Saved pet data that can be restored when the application is relaunched.
5. Progression features such as XP, coins, and levels, supported by a food shop and mini-game.
6. A draggable, always-on-top floating desktop pet with a right-click menu.

These are the outcomes of the implemented application. This report does not provide measured performance figures, test logs, screenshots of each test, or quantitative user-evaluation results; therefore, no numerical success rate is claimed.

7.2 Functional Test Cases

Test case	Procedure	Expected result
1. Pet adoption	Open the application and create/name a pet.	Pet is created and displayed.
2. Care action	Select feeding, playing, sleeping, or bathing.	Relevant pet attributes update.
3. Save state	Perform an action and leave the application.	Latest pet state is stored locally.
4. Restore state	Close and relaunch the application.	Previously saved pet information is restored.
5. Real-time stats	Leave the application running	Stats change at the pet-

	for some time.	specific rates.
6. Progression	Perform supported interactions.	XP, coins, or levels update as designed.
7. Desktop-pet mode	Click minimize, drag the pet, and right-click it.	Pet floats on top of windows, can be dragged, and shows the menu.

Table 1. Functional test cases. These are recommended checks for validating the project; they should be marked as passed only after they have been executed against the application.

7.3 Limitations

1. LocalStorage data is stored in the Electron application's data directory on a single device and does not automatically synchronize across devices.
2. Deleting the application's data or using the reset option removes the saved pet state.
3. Stats change only while the application is running; the time elapsed while the application is closed is not simulated.
4. No quantitative performance or usability measurements were conducted.

Cloud accounts, native notifications, achievements, and smarter pet behaviour are listed as future scope.

Chapter 08: Conclusion & Future Scope

8.1 Conclusion

The project, Mochi: A Persistent Electron-Based Desktop Virtual Pet System with Real-Time State Management, combines interactive gameplay with persistent desktop-application design. Users can adopt one of four pet types and care for it through feeding, playing, sleeping, and bathing, while the application tracks hunger, energy, happiness, cleanliness, and health. HTML, CSS, and JavaScript provide the interface and behaviour, Electron packages them as a cross-platform desktop application, and LocalStorage preserves data between launches.

The main contributions are the combination of saved state with real-time, pet-specific stat updates, and the use of Electron's main/renderer architecture and IPC to provide a draggable, always-on-top desktop pet. All artwork is generated programmatically as SVG, and progression through XP, coins, levels, a food shop, and a mini-game supports engagement. Overall, the project demonstrates practical concepts in desktop application development, event handling, state management, IPC, persistent storage, and programmatic graphics.

8.2 Future Scope

The following enhancements could expand the application's functionality:

1. **Additional pet types and animations:** add new species beyond the current four, with their own SVG art and trait modifiers.
2. **Customization:** provide options for names, accessories, themes, or visual appearance.
3. **More mini-games:** add further activities beyond "Catch the Ball" that reward users with XP or coins.
4. **User accounts and cloud synchronization:** make pet data available across devices and sessions through a backend service.
5. **Notifications:** send native desktop notifications to remind users when the pet needs attention.
6. **Smarter pet behaviour:** introduce richer responses, moods, or behaviour patterns based on the pet's history and condition.
7. **Offline time simulation:** store a timestamp on exit and apply the corresponding stat changes when the application is relaunched.

8. **Achievements:** add achievements to reward milestones such as levels, care streaks, and mini-game scores.

These enhancements would build on the existing Electron foundation. Cloud synchronization and account management would require additional server-side services and appropriate data-security measures.

REFERENCES

[1] MDN Web Docs, “HTML: HyperText Markup Language.” https://developer.mozilla.org/en-US/docs/Web/HTML

[2] MDN Web Docs, “CSS: Cascading Style Sheets.” https://developer.mozilla.org/en-US/docs/Web/CSS

[3] MDN Web Docs, “JavaScript.” https://developer.mozilla.org/en-US/docs/Web/JavaScript

[4] MDN Web Docs, “Window: localStorage Property.” https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

[5] MDN Web Docs, “Date.” https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date

[6] Electron, “Electron Documentation.” https://www.electronjs.org/docs

[7] electron-builder, “electron-builder Documentation.” https://www.electron.build/

[8] Node.js, “Node.js Documentation.” https://nodejs.org/en/docs

[9] MDN Web Docs, “SVG: Scalable Vector Graphics.” https://developer.mozilla.org/en-US/docs/Web/SVG

[10] “Mochi_VP: Mochi, a desktop virtual pet built with Electron” (source code). https://github.com/RiddhiInLoop/Mochi_VP